

임베디드SW

전자정보대학 정보통신공학부
최 민

ARM History

● 최초의 **ARM** 프로세서

- 영국 캠브리지, Acorn Computers Limited 社에서 개발

● **ARM**

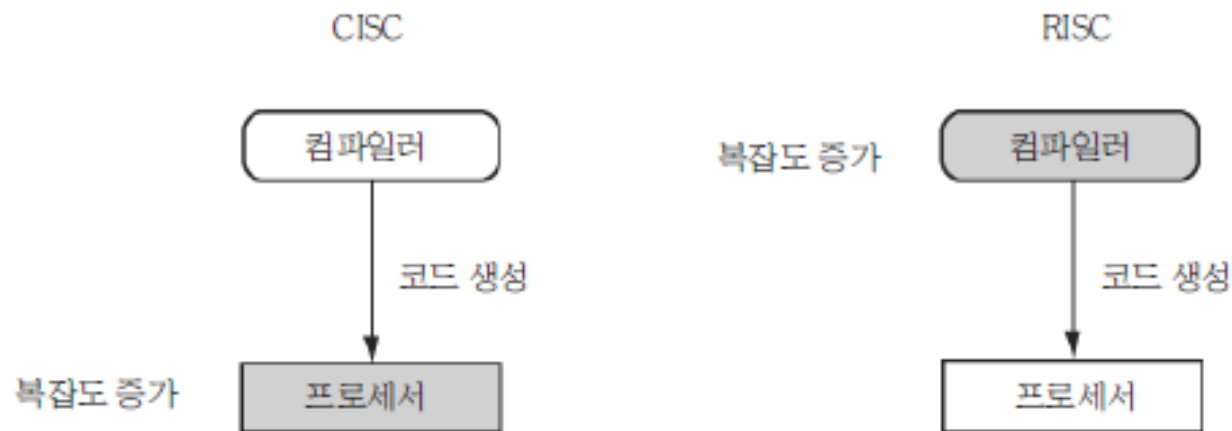
- 32-bit RISC구조 (Reduced Instruction Set Computer)
- Acorn RISC Machine → Advanced RISC Machine

● 연혁

- 1985년 ARM1 프로토타입 출시
- 2001년 말까지 20억개 이상 판매
- 19년동안 판매한 누적 판매량: 120억개
- 2009년 ARM 기반 모바일 칩(퀄컴, 삼성) 출하량 : 약 39억개
- 현재, 전세계 휴대폰 95% 이상이 ARM 코어 프로세서 탑재

● RISC 구조

- 매우 빠른 속도로 한 클럭 안에 실행될 수 있는 간결하면서도 막강한 명령어들을 가지고 있는 아키텍처
- 하드웨어에 의해 수행되는 명령어들의 복잡도를 줄이는 것을 목표
- 컴파일러 의존적
 - CISC는 하드웨어 복잡도에 초점, RISC는 컴파일러 복잡도에 초점



CISC/RISC 비교

정 의

- 매우 빠른 속도로 한 클럭 안에 실행될 수 있는
- 간결하면서도 막강한 명령어들을 가지고 있는 아키텍처

목 표

H/W에 의해 수행되는 복잡한 명령어를 줄이는 것

구 분	RISC	CISC
명 령 어	간단한 명령어를 조합하여 복잡한 명령어 구현	명령어 길이가 가변적이고 명령어 실행에 많은 사이클 소모
파이프라인	병렬로 실행가능한 작은 단위로 분리	미니 프로그램(마이크로코드) 필요
레지스트	데이터와 주소가 저장 가능한 거대한 레지스터군 보유	특정목적을 위한 전용 레지스트
로드스토어 아키텍처	로드명령어: 레지스트 읽을시 스토어명령어: 레지스트 저장시	
클 력	빠른 클럭으로 동작	복잡하고 낮은 클럭으로 동작
초 점	컴파일러 복잡도	하드웨어 복잡도

ARM 고유 핵심기능

● Instructions

- reduced set/single cycle/fixed length(32bit)

● Pipeline

- decode in one stage/no need for microcode

● Registers

- a large set of general-purpose registers

● Load/store architecture

- data processing instructions apply to registers only; load/store to transfer data from memory

● 명령어 파이프라인

- 초기엔 3-stage pipeline, 현재는 5-stage pipeline

Why we study ARM?

● 울트라 모바일 디바이스(UMD)

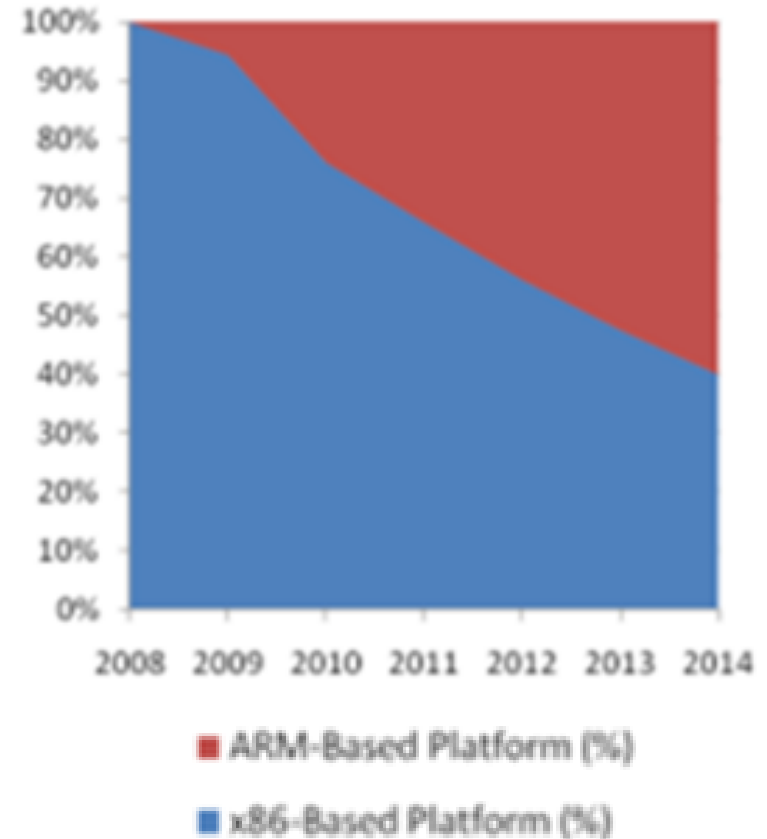
- 로우엔드 랩탑(low-end laptop)
- 넷북, MID, 스마트북, UMPC

● 시장 격돌

- Intel Atom 아키텍처
 - 인텔은 2000년대 중반 ARM과 결별
 - PXA family를 Mavell에 넘김
- ARM Cortex 아키텍처

● ABI 리서치, 미국 시장조사 기관

- 향후 ARM 기반 플랫폼이 x86 플랫폼을 넘어설 것



Why we study ARM?

● 다윗(**ARM**)과 골리앗(**Intel**)의 싸움?

- 2009년 기준 인텔 매출: 251억\$, 직원 수: 7만8900명
- 2009년 ARM 매출: 4억8000만\$(50분의 1 수준), 직원 수: 1700명

● Intel

- MS (Windows/Windows CE)와 협력

● ARM

- 반도체 설계 기술을 공급받는 모든 업체와 협력
- Linux, Apple Iphone/iPad, Google Android, Nokia Symbian, 퀄컴, 삼성, LG, 엔비디아, HP, 델, 에이서, 레노보, Analog Devices, Atmel, Cirrus, Fujitsu, IBM, Infineon, Mitsubishi, Motorola, National Semiconductor, NEC, Philips, Sharp, ST Microelectronics, Texas Instruments, Toshiba, NEC, Philips, Sharp, ST Microelectronics, Texas Instruments, Toshiba 등 660개 협력업체

Why we study ARM?

● CES와 MWC

- 출품된 MID, 스마트폰, 넷북, 스마트북 등 모바일 인터넷 기기
- 상당수가 ARM의 Cortex 기반
- ARM의 CPU 코어는 다양한 칩셋과 OS의 탑재가 가능해 꾸준히 매출이 증가하고 있다

● 주요 리서치 기관 (**ABI** 리서치, **IDC**, 가트너 등)

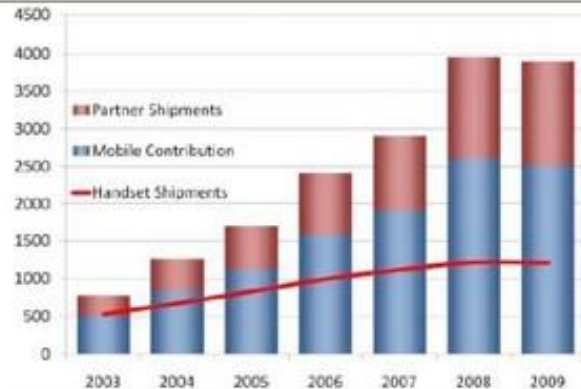
- 2014년까지 모바일 컴퓨팅 시장 규모가 5억대 규모로 성장할 것으로 관측, ARM의 순항은 지속될 전망

Why we study ARM?

- 2009년 전체 반도체 시장이 20% 정도 감소한 반면,
 - 2008년 판매제품 물량 : 약 40억 개
 - 2009년 : 39억 개 (2% 정도 감소)

ARM Powers the Mobile World

- >12 Billion ARM Powered® chips in mobile devices
- 2.5 Billion ARM Powered chips shipped in 2009
- Average phone has > 2.0 ARM chips
- Scalable performance from ULCH to PC
- CAGR ~ 40% of smartphone predicted until 2013



Industry units (ex-memory) down ~20%
ARM FY09: 3.9bn units (down ~2%)



Why we study ARM?

- 스마트폰 한대에 다양한 종류의 **ARM** 프로세서 칩셋 사용
 - 많게는 대당 5~7개 ARM core 사용

ARM Powered Smartphone



RISC 장단점

● 장점

- 손쉽게 성능 최적화 가능
- 고정 길이의 명령 세트는 더욱 빠른 실행을 허용
- 파이프라인을 더욱 효율적으로 사용
- 더 빠른 클럭 속도를 허용하며, 축소 명령 세트로 인해 칩 다이 크기가 더욱 작아짐

● 단점

- 코드 크기(size)가 커지는 현상
- 32-bit RISC 기계에서, 모든 단일 명령어는 코드 공간의 4 bytes를 필요로 함
- ARM “Thumb” 기술은 이에 대한 solution을 제공

- 임베디드 시스템에서 가장 인기있는 **ARM** 코어

- ARM7TDMI, ARM9EJS

- **TDMI**

- Thumb

- Debug support

- 하드웨어 디버그 기술, 소프트웨어 엔지니어들은 프로세서가 코드를 실행하는 동안 내부에서 어떤 일들이 일어나는지 확인가능

- Multiplier (64-bit result)

- In-Circuit Emulator interface

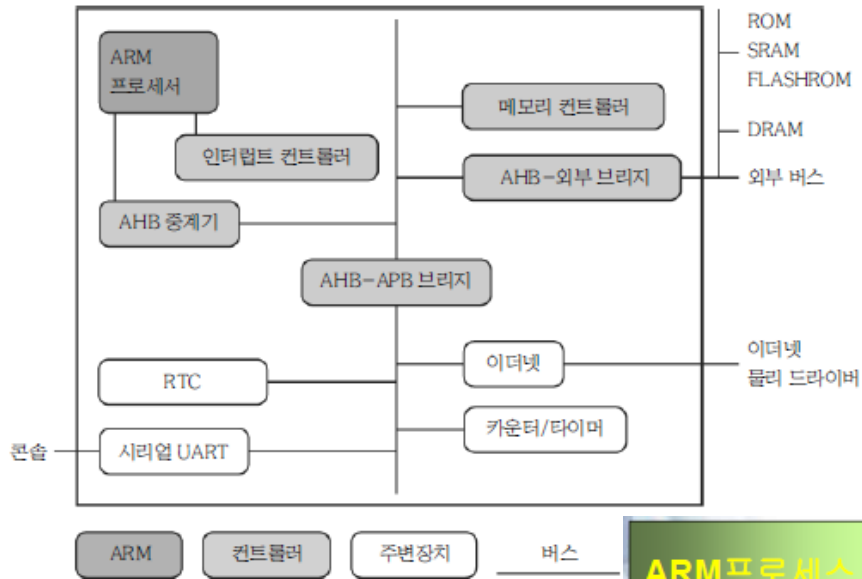
- **EJS**

- Enhanced instruction

- Jazelle (JVM)

- Synthesizable version (source code version for EDA tools)

ARM 하드웨어 임베디드 시스템 아키텍처



ARM프로세서

임베디드 디바이스를 제어하는 역할
코어와 주변컴포넌트(메모리관리장치, 캐시 등)으로 구성

컨트롤러

주요한 기능을 가진 블록을 조정(인터럽트 컨트롤러, 메모리컨트롤러)

주변장치

칩 외부로의 입.출력 기능을 제공

임베디드 디바이스의
응용 분야에 영향을
미친다.

버 스

주변 장치간의 통신

ARM 버스 기술

특징

x86 PC를 위해 설계된 것과는 다른 버스 체계 사용

- .PCI → 버스가 칩 외부에 존재하여 기계적, 전자적으로 연결
- .임베디드 디바이스 → 칩 내부에 버스가 존재하여 주변 장치와 연결

디바이스

- .버스마스터: 동일한 버스를 통해 초기화된 데이터를 전송할 수 있는 논리 장치
- .버스슬레이브: 버스마스터로부터 전송요청이 있을 때에만 동작하는 논리장치

아키텍처 단 계

- .물리적 단계 : 전기적 특성과 버스폭을 담당
- .프로토콜 처리단계

메모리 구성

● ARM7

- 폰 노이만 아키텍처에 기초
- Code / Data 를 위한 공유된, 단일 메모리 공간
- 선형 32-bit 주소 영역 (4 GByte)

● ARM9

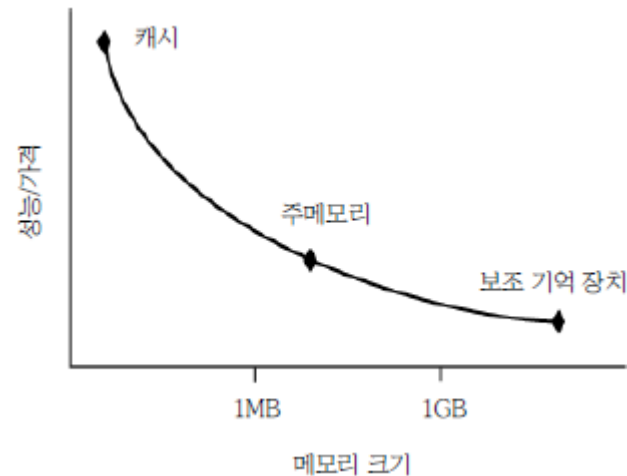
- Harvard architecture 지원
- Code / Data를 위한 개별적인 메모리 포트
- Code / Data 에 대한 동시 액세스 제공

메모리 구성

메모리 계층구조

- 캐시 : 가장 빠른 메모리, 물리적으로 ARM 프로세서 코어에 가장 가까이 위치
- 보조 기억 장치 : 가장 느린 메모리, 프로세서에서 가장 멀리 떨어져 있다.

메모리가 프로세서에 가까울수록 : 비용↑ 용량↓



메모리의 트레이드오프(trade-off)

메모리 구성

- 한 명령어 실행을 위한 메모리 접근 횟수
 - 32비트 ARM 명령어와 16비트폭의 메모리칩을 사용하는 캐시가 없는 경우? 16비트 Thumb 명령어를 사용한다면?
 - 16 비트폭의 메모리에 Thumb 명령어를 사용하면?

명령어 길이	8 비트 메모리	16 비트 메모리	32 비트 메모리
ARM 32비트	4사이클	2사이클	1사이클
Thumb 16비트	2사이클	1사이클	1사이클

메모리 구성

구 분	특 징
ROM	▶ 제품을 출시할 때 영구적으로 저장되어 다시 변경할 수 없는 이미지를 포함하고 있는 메모리 ▶ 업데이터나 수정이 필요없는 대량생산품 또는 부트코드 저장에 사용
플래시롬	▶ 읽고 쓰기가 가능하며, 대량의 보조기억 장치의 대체품으로 사용 ▶ 데이터를 쓰는데 느리기 때문에 동적인 데이터를 저장하는데 적합하지 않음
DRAM	▶ 가장 일반적으로 사용되는 RAM으로 리프레시 작업필요 ▶ 다른 RAM에 비해 메가바이트당 가장 저비용
SRAM	▶ DRAM보다 속도가 빠르고 리프레시 작업 필요 없음 ▶ 실리콘 공간을 더 많이 필요로 하며 고가
SDRAM	▶ DRAM의 하위범주의 메모리 ▶ 기존의 메모리보다 훨씬 빠른 속도로 동작

주변장치

기능

칩 외부에 다른 디바이스나 센서를 연결해서 칩의 입.출력 기능을 수행
모든 ARM 주변장치는 메모리에 매핑되어 있음

1.3.4.1 메모리 컨트롤러

다양한 유형의 메모리를 프로세스 버스에 연결해 주는 역할

ex) DRAM을 사
용할 때, 이를 액세스하기
전에 메모리 타이밍과
리프레시 비율을 설정

1.3.4.2 인터럽트 컨트롤러

- ▶ 표준인터럽트 컨트롤러 **디바이스 비트맵 레지스터를 읽고 판단**
: 외부 디바이스가 서비스를 요청하면 그 인터럽트 신호를 프로세스 코어에게 보냄
- ▶ 벡터 인터럽트 컨트롤러 **현재 인터럽트보다 우선순위가 높은 경우에만 인터럽트 신호를 보낸다**
: 인터럽트에 우선순위를 부여하고 어떤 디바이스가 인터럽트를 발생하였는지에 대한 결정을 단순화

인텔 vs. ARM

- <http://byline.network/2016/08/1-305/>